Parity Solver

parity_solver.py

Description & Operating Instructions

1. Overview

parity_solver.py is a command-line tool for analysing Collatz paths via the path equation. Given a parity vector — a sequence of Odd (O) and Even (E) steps — the program determines the three parameters that completely characterise the path and then solves the resulting linear Diophantine equation for all positive integer seed pairs (N, M).

The central equation is the Collatz master invariant:

$$(3^n \cdot N + f) / 2^m = M$$

Rearranged into standard Diophantine form:

$$2^m \cdot M - 3^n \cdot N = f$$

Here **N** is the starting seed, **M** is the value after m steps, **m** is the total number of steps, **n** is the number of odd steps, and **f** is the accumulated residue computed from the O-step positions.

Because gcd(2^m, 3^n) = 1 for all m and n, the equation always has integer solutions. The program finds the complete parametric family, identifies the residue class that the path belongs to, checks for loop solutions (M = N), and reports the M = 1 seed where it exists.

2. Key Concepts

2.1 The Parity Vector

The parity vector is a string of length m, one character per Collatz step, recording whether the current value was odd or even at that step. Each character is either O (odd step, applying (3x+1)/2) or E (even step, applying x/2). The vector fully determines m, n, and f.

2.2 Path Parameters

Symbol	Name	Definition
m	Total steps	Length of the parity vector.
n	Odd steps	Count of O characters in the vector.
f	Accumulated residue	Built by the recurrence: at each O-step at 1-indexed position i, f ← 3f + 2^(i−1). Starts at f = 0.
b	Residue class	The unique value in [0, 2^m) such that any seed N ≡ b (mod 2^m) follows this exact path. Equal to the minimum valid seed mod 2^m.
gap	Group indicator	gap = $2^m - 3^n$. Positive → Group A (descent possible). Negative → Group C (ascent guaranteed). Zero is impossible.

2.3 The General Solution

Since gcd(2^m, 3^n) = 1, the extended Euclidean algorithm finds a particular solution (M_0, N_0). The full family of integer solutions is then:

$$M = M_0 + 3^n \cdot t$$
$$N = N_0 + 2^m \cdot t \qquad (t \in \mathbb{Z})$$

Both M and N are positive for all t ≥ t_min. Crucially, because the step size for N is exactly 2^m, every solution in the family shares the same residue class: N ≡ b (mod 2^m). The path is therefore self-consistent across the entire solution family.

2.4 Loop Detection

Setting M = N in the general solution requires −gap·t = $N_0 - M_0$. A loop exists if and only if gap divides ($N_0 - M_0$) and the resulting t yields a positive value. The only loops found in all tested paths are the two known attractors: M = N = 1 (path OEOEOE…) and M = N = 2 (path EOEOEO…).

2.5 The δ Quantity

The optional **δ** column displays δ(N) = f/N − gap for each solution. This is the signed distance from the loop threshold: δ < 0 means M < N (descent), δ = 0 is a loop, and δ > 0 means M > N (ascent). For Group A paths with large N, δ converges toward −gap.

3. Requirements

- **Python 3.6 or later** (uses f-strings and math.ceil).

- No third-party packages required. The standard library is sufficient.
- The script is a single file: `parity_solver.py`.

4. Usage

4.1 Basic Syntax

```
python parity_solver.py <PATH> [OPTIONS]
```

where `<PATH>` is the parity vector. Both O/E and 1/0 notation are accepted and may be freely mixed.

4.2 Command-Line Options

Option	Short	Default	Description
`--solutions N`	`-s N`	8	Number of positive (M, N) solution pairs to display.
`--delta`	`-d`	off	Add a δ = f/N − gap column to the solutions table.
`--help`	`-h`	—	Show the built-in help message and exit.

4.3 Input Format

The path vector is case-insensitive. The following are all equivalent representations of the same path:

```
OOEE
ooee
1100
O0O0    ← mixed notation also accepted
```

> **Note**
> Any character other than O, E, 1, or 0 raises an error.
> An empty string raises an error.

5. Examples

5.1 Basic path analysis

```
python parity_solver.py OOEE
```

Analyses the path OO EE (two odd steps then two even steps). Displays parameters, the Diophantine equation, the general solution family, loop analysis, the first 8 solution pairs, and the M = 1 seed if it exists.

5.2 Binary notation

```
python parity_solver.py 1100
```

Identical to the previous example. Binary 1 = odd step, 0 = even step.

5.3 Requesting more solutions

```
python parity_solver.py OOOEEE --solutions 20
```

Displays the first 20 positive solution pairs for the path OOO EEE.

5.4 Displaying the δ column

```
python parity_solver.py OOEEOOE --delta
```

Appends a δ(N) column to the solutions table, showing how far each seed is from the loop threshold.

5.5 Combining options

```
python parity_solver.py OEOEOE --solutions 10 --delta
```

Shows 10 solutions with δ. For this path (the known N = 1 attractor) the first entry will report a confirmed loop at M = N = 1 with δ = 0.

5.6 Long paths

```
python parity_solver.py OOEEOOEEOOEEOOE --solutions 5 --delta
```

The solver handles paths of any length. Arithmetic is performed with Python arbitrary-precision integers, so there is no overflow at large m.

6. Reading the Output

6.1 Parameters block

Lists m, n, f, b, and gap for the path. The group label in brackets after gap indicates Group A (descent possible) or Group C (ascent guaranteed).

6.2 Equation block

Displays the specific Diophantine equation for this path in both forms: the subtraction form used for solving, and the division form matching the master invariant.

6.3 General solution block

Shows M_0, N_0 (the canonical particular solution with M_0 in $[0, 3^n)$) and the step sizes 3^n and 2^m. The minimum t value for positive solutions is shown in the heading.

6.4 Loop analysis block

Reports whether M = N is achievable. If a loop exists, the seed value and a $\delta = 0$ confirmation are shown. If no loop exists, the divisibility obstruction is stated explicitly: the value $N_0 - M_0$ and the gap, and whether the former is divisible by the latter.

6.5 Solutions table

Each row is one valid (t, N, M) triple with M/N ratio and a verification tick. The ratio M/N converges toward $3^n/2^m$ as $t \to \infty$, reflecting the asymptotic descent toward the attractor. An optional δ column is appended when --delta is used.

6.6 Special seeds block

Reports the unique N that produces M = 1 under this path, computed as $N = (2^m - f) / 3^n$. If this value is not a positive integer, or does not belong to residue class b, a message explains why no such seed exists.

7. Interpreting Results

Group A paths (gap > 0)

The M/N ratio is less than 1 for the minimum seed and converges further below 1 as N grows. This means the orbit descends: M < N for all sufficiently large seeds in this residue class. The δ values will all be negative for large t.

Group C paths (gap < 0)

M > N for every solution in the family — the orbit ascends regardless of the seed. The M/N ratio converges toward $3^n/2^m > 1$ from above. No loop is possible in a Group C cell.

Loop confirmed

A loop at M = N means the Collatz orbit returns to its starting value after exactly m steps. In all tested paths the only confirmed loops are M = N = 1 (path OEOEOE…) and M = N = 2 (path EOEOEO…), corresponding to the trivial {1, 2, 4} cycle.

M = 1 seed

The seed $N = (2^m - f) / 3^n$, when it is a valid positive integer in residue class b, is the unique starting point that lands exactly at 1 after m steps under this path. This seed is the entry point into the {1, 2, 4} attractor cycle.

8. Quick Reference

Command	What it does
`python parity_solver.py OOEE`	Analyse path OOEE, show 8 solutions.
`python parity_solver.py 1100`	Same as above using binary notation.
`python parity_solver.py OEOEOE`	Analyse the N=1 attractor path, confirm loop.
`python parity_solver.py EOEOEO`	Analyse the N=2 attractor path, confirm loop.
`python parity_solver.py OOO`	Group C example: all odd steps, M/N > 1.
`python parity_solver.py OOOEEE -s 20`	Show 20 solutions for path OOOEEE.
`python parity_solver.py OOOEEE -d`	Show δ column for path OOOEEE.
`python parity_solver.py OOEEOOE -s 12 -d`	12 solutions with δ for path OOEEOOE.